Chapter 2

Asymptotics for cryptographers, and the Factoring Assumption

> There are two kinds of cryptography in this world: cryptography that will stop your kid sister from reading your files, and cryptography that will stop major governments from reading your files.
>
> Bruce Schneier, 1996

Introduction

Our goal is to build an encryption scheme Π for which we can prove a guarantee like:

If problem X is hard, then Π is secure.

This is the realm of **computational security**: the security of our cryptographic schemes is based on assumptions about which problems are computationally feasible and which are computationally infeasible. By their very nature, computational security guarantees would fail against an adversary who had infinite compute. In contrast, perfect secrecy is an example of **information-theoretic security**: no computational assumptions are required and guarantees hold even against infinitely powerful adversaries. The security of the one-time pad is absolute, if used correctly (uniformly random keys used once). Computational security is not.

We need to build up a lot of machinery to formalize a guarantee like the one above.

- **Hardness assumptions**: What does it mean for a "problem to be hard"?
- **Security definitions**: What does it mean for an "encryption scheme to be secure"?
- **Reductions**: How do we connect the two?

That's the goal of the next few weeks. First, we'll start with hardness assumptions and reductions, focusing specifically on the conjectured hardness of *factoring*. We want to formalize statements that mean: "No efficient algorithm can solve typical instances of a problem except

with negligible probability." To make this concrete, we need to define "efficient algorithm" and "extremely small probability."

Later we'll construct a (public key!) encryption scheme from factoring, and prove that it enjoys a basic security property (a form of *indistinguishability*). Finally, we'll define a seemingly much stronger security guarantee (*semantic security*) and show that the basic security property is actually equivalent.

Why computational security?

Why abandon perfect secrecy and settle for the qualitatively weaker computational security guarantee? Because perfect secrecy has inherent limits that are unacceptable in practice. Here's an excerpt from Katz-Lindell [Katz and Lindell, 2021].

Necessity of the Relaxations

Computational secrecy introduces two relaxations of perfect secrecy: first, secrecy is guaranteed only against efficient adversaries; second, secrecy may "fail" with small probability. Both these relaxations are essential for achieving practical encryption schemes, and in particular for bypassing the negative results for perfectly secret encryption. We informally discuss why this is the case. Assume we have an encryption scheme where the size of the key space $\mathcal{K}$ is smaller than the size of the message space $\mathcal{M}$. (As shown in the previous chapter, this means the scheme cannot be perfectly secret.) Two attacks apply regardless of how the encryption scheme is constructed:

- Given a ciphertext c, an adversary can decrypt c using all keys $k \in \mathcal{K}$. This gives a list of all the messages to which c can possibly correspond. Since this list cannot contain all of $\mathcal{M}$ (because $|\mathcal{K}| < |\mathcal{M}|$), this attack leaks *some* information about the message that was encrypted.

  Moreover, say the adversary carries out a known-plaintext attack and learns that ciphertexts $c_1, \ldots, c_\ell$ correspond to the messages $m_1, \ldots, m_\ell$, respectively. The adversary can again try decrypting each of these ciphertexts with all possible keys until it finds a key k for which $\mathsf{Dec}_k(c_i) = m_i$ for all i. Later, given a ciphertext c that is the encryption of an unknown message m, it is almost surely the case that $\mathsf{Dec}_k(c) = m$.

  Brute-force attacks like the above allow an adversary to "succeed" with probability ≈ 1 in time $\mathcal{O}(|\mathcal{K}|)$.

- Consider again the case where the adversary learns that ciphertexts $c_1, \ldots, c_\ell$ correspond to messages $m_1, \ldots, m_\ell$. The adversary can *guess* a uniform key $k \in \mathcal{K}$ and check whether $\mathsf{Dec}_k(c_i) = m_i$ for all i. If so, then, as above, the attacker can use k to decrypt anything subsequently encrypted by the honest parties.

  Here the adversary runs in constant time and "succeeds" with nonzero probability $1/|\mathcal{K}|$.

Nevertheless, by setting $|\mathcal{K}|$ large enough we can hope to achieve meaningful secrecy against attackers running in time much less than $|\mathcal{K}|$ (so the attacker does not have sufficient time to carry out a brute-force attack), except possibly with small probability on the order of $1/|\mathcal{K}|$.

2.1 Motivating discussion for the asymptotic approach

Informally, a problem is hard if: **no efficient algorithm can solve random instances of the problem except with extremely small probability.** In this class, hardness is always going to be stated as an **asymptotic**, **average case** property.

We'll formalize asymptotics, efficient algorithms, and extremely small probabilities in the following sections. For now, recall from your algorithms class that asymptotic analysis de-

scribes the behavior of functions as their inputs grow arbitrarily large.

This section presents a concrete problem that is conjectured to be hard: factoring.

2.1.1 Average case hardness vs worst case hardness

Traditional complexity theory (i.e., P vs NP; your algorithms class) is primarily concerned with **worst case** analysis of algorithms. "What is the slowest an algorithm $\mathcal{A}$ can run, over all possible inputs?"

An example of an NP-hard problem is the Travelling Salesman Problem (TSP): given a complete weighted graph $G = (V, E)$, find the shortest cycle that visits every vertex exactly once. When we conjecture that the Travelling Salesman Problem (TSP) is hard, we mean hard in the worst case: for any algorithm $\mathcal{A}$ solving TSP, there are graphs on which $\mathcal{A}$ is inefficient. Equivalently: for any efficient algorithm $\mathcal{A}$, there are graphs on which $\mathcal{A}$ errs.

Worst case hardness says nothing about how hard "typical" instances of a problem are. It is entirely possible that both of the following are true: (i) TSP is hard in the worst case; (ii) there is an efficient algorithm that solves every instance of TSP that has ever come up in a real-world application.

In cryptography, we want security to hold in typical, everyday uses of our systems. We need problems to be hard in the **average case**.[1] For this to be well-defined, we need to specify the *distribution over problem instances* with respect to which we're taking the average. Moreover, we generally need the hard distribution to be *sampleable*: there is an efficient algorithm to generate a sample from the hard distribution over problem instances.

2.1.2 Why asymptotic analysis? Tunable hardness, clean theory

A core challenge of cryptography is the asymmetry between the users we're trying to protect and the adversaries attacking them. The users are you and me, or a dissident in an authoritarian regime using WhatsApp on a decade-old phone with spotty cellular service. The adversary is the secret police of that authoritarian regime, or the NSA—the United States' cryptographic agency and the world's largest employer of mathematicians. To be effective, there must be a corresponding asymmetry in the cryptographic scheme. Effective cryptography has to be easy (read: efficient) for the users to use, and hard (read: infeasible) for the adversaries to break.

We could try to bound the computing resources available to the users and adversary. (The following is purely illustrative, and not actually how security is quantified in practice.) For the user, maybe we are okay requiring at least 1 CPU minute per message sent (this would be very slow!). For the adversary, maybe we're okay if it takes the NSA a year to break our scheme. For a very crude estimate, divide the USA intelligence services' budget ($\approx \$30 \times 10^9$ per year) by the cost of cloud compute (≈ 0.01 per CPU hour on AWS) times a safety factor (10^9) to get a number of CPU hours that the NSA is unlikely to be able to exceed in a single year (3×10^{21}).[2]

[1] Problems that are hard on average are also hard in the worst case. The reverse is not generally true.

[2] This estimate is only meant to be illustrative. It is very crude, especially the use of commodity pricing for general CPUs instead of the cost of computing on special-purpose hardware.

In the above example, we want a factor of, say, 10^{23} difference in the compute required to use the scheme and to break it. More generally, we want **tunable hardness**. We want a dial we can turn. **When we dial up the hardness, it makes the scheme a little harder for the users, and a lot harder for the attackers.**

Suppose the key (the only thing the adversary doesn't know!) to our cryptographic scheme is $k \in \{0,1\}^n$. Honest parties must, at the very least, read in the whole key. This takes n operations. Attackers can, at the very least, try out every key.[3] This takes at most 2^n iterations. Alternatively, an attacker can randomly guess the key, with probability 2^{-n} of getting it right on the first try.

Ideally, we want our security to scale in the same way: to make the scheme twice as secure, we increase the key length by 1. More generally, we want that: **as the key grows, the compute required to break the scheme grows much faster than the compute required to use the scheme.** So cryptography naturally requires studying the efficiency of computational problems as their input (the key) grows. This is the realm of **asymptotic analysis of algorithms**.

In this example, the key length n is the **security parameter**: the dial that we use to tune how secure our schemes are. More generally, the security parameter controls the size of the relevant objects. For the factoring assumption below, n will be the bit length of each prime factor.

2.2 The Factoring Assumption, informally

The factoring problem is simple to describe: given a composite number $N \in \mathbb{N}$ as input, output a **non-trivial divisor** d: an integer $1 < d < N$ such that $d \mid N$. There is no known efficient algorithm for factoring large integers N.[4] It is widely believed that there is no efficient algorithm.

We want to turn the conjectured hardness of factoring into a formal **hardness assumption**. For the reasons described above, a well-specified hardness assumption requires us to specify a *distribution over problem instances* (for average-case hardness) and these instances should be *of arbitrary size* (for asymptotic analysis).

For factoring, the problem instance is the positive integer N. The size of the problem is $\|N\|$, the number of bits it takes to represent the number N in binary with a leading bit 1. The integers N that require n bits to represent form the set $[2^{n-1}, 2^n)$. Hence, $\log N < \|N\| \leq \log N + 1$.

For example, a distribution over n-bit composite numbers, denoted $\mathcal{D}_n$, is a probability distribution supported on the composite numbers in $[2^{n-1}, 2^n)$. To support instances of arbitrary size, we need to define an infinite sequence of distributions $\mathcal{D}_n$, for an infinite sequence of problem sizes n. For example, we might take $\mathcal{D}_n$ to be any of the following:

(a) A uniformly random composite number in the set $[2^{n-1}, 2^n)$

(b) 2^{n-1} with probability 1

(c) $2^{n-1} + 2k$ with probability $1/2^k$ for $1 \leq k \leq 30$, and 2^{n-1} with probability $1/2^{30}$ for sufficiently large n (i.e., $2^n > 2^{n-1} + 60$)

[3] ...assuming the attacker can check whether candidate key is correct. See Exercise 2.8.

[4] ...except using quantum computers! We'll discuss quantum in a future lecture.

Once we fix the (sequence of) distributions $\mathcal{D}_n$, the **Factoring Problem** is the following:

- INPUT: $N \leftarrow \mathcal{D}_n$
- OUTPUT: A non-trivial divisor d: an integer $1 < d < N$ such that $d \mid N$

Whether factoring is hard or easy depends on the distribution $\mathcal{D}_n$. For some distributions factoring is very easy. For example, in the distributions above:

(a) For large n, 2 is a divisor with probability approximately $1/2$, 3 with probability approximately $1/3$, 5 with probability approximately $1/5$, and so on. Trying a few small primes succeeds with significant probability. Even just checking whether N is even succeeds with probability close to $1/2$.

(b) 2 is always a divisor: $2 \mid 2^{n-1}$

(c) 2 is always a divisor: $2 \mid 2^{n-1} + 2k$

In contrast, factoring products of two distinct, uniformly sampled primes of equal bit length is conjectured to be hard. Two relevant classes of numbers are:

- **Semiprime**: $N = pq$ for primes p, q. If p and q are both n bits, then N is either $2n-1$ or $2n$ bits.
- **Blum integers**: $N = pq$ for distinct primes $p, q \equiv 3 \bmod 4$.

For the hard distribution in Section 2.7, we will sample two distinct n-bit primes, each congruent to 3 mod 4, and multiply them. Here n indexes the bit length of the factors; in the earlier examples it indexed the bit length of N itself. Either convention supports asymptotic analysis, since these lengths differ by at most a factor of 2.

We want to say that for this distribution over Blum integers N, "No efficient algorithm can solve the Factoring problem except with extremely small probability." We have to include "except with extremely small probability," because we can't prevent an algorithm from getting lucky, say, by just guessing a random number and hoping it's equal to p or q.

To make this concrete, we need to define "efficient algorithm" and "extremely small probability." We'll do that in the following sections. The formal statement of the factoring assumption is in Section 2.7.

2.3 Efficient algorithms: probabilistic polynomial time (PPT)

We will consider algorithms efficient if they run in polynomially-bounded time on all inputs. We call randomized algorithms with such a guarantee **probabilistic polynomial time (PPT)**.

2.3.1 Preliminaries: Big O notation and polynomials

We use **asymptotic analysis** as our foundation. Asymptotic analysis describes the behavior of functions as the input grows $\to \infty$. Let f and g be non-negative functions $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$.

Definition 2.1 (Big O notation)**.** *We write $f(n) = O(g(n))$ if there exists $C \geq 0$ such that there exists $n_0 \geq 1$ such that for all $n \geq n_0$,*

for all sufficiently large n

$$f(n) \leq C \cdot g(n).$$

The phrase **"for all sufficiently large n"** is a very useful shorthand for "there exists $n_0 \geq 1$ such that for all $n \geq n_0$." We will use the shorthand from now on.[5]

Definition 2.2. *A function $f : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is **polynomially-bounded** ("poly" for short) if there exists $c \geq 0$ such that $f(n) = O(n^c)$. We denote a generic polynomially-bounded function by* poly.

Example 2.1. $7 = O(1)$, $3n+2 = O(n)$, and $n \log n = O(n^2)$. Also, $4n^2+3n+1 = O(n^2)$, and $\log n = O(n)$. But $n^2 \neq O(n)$, and $2^n \neq O(n^3)$.

7, $n^3 + 2n$, and $n \log n$ are all polynomially-bounded. So are $\sqrt{n} = O(n^{1/2})$, $\log n = O(n^{0.00001})$, and even $n^{100} = O(n^{100})$. Neither 2^n nor $n!$ is polynomially-bounded.

Definition 2.3 (Probabilistic polynomial time (PPT))**.** *For a randomized algorithm $\mathcal{A}$, let $\mathsf{Time}_{\mathcal{A}}(n)$ be the maximum running time of $\mathcal{A}$ on inputs $x \in \{0,1\}^n$ of length n.*[6] *$\mathcal{A}$ runs in **probabilistic polynomial time (PPT)** if $\mathsf{Time}_{\mathcal{A}}(n)$ is polynomially-bounded.*

For example, checking whether a string $x \in \{0,1\}^n$ is all zeroes takes $O(n)$ time. As the input size n grows, the runtime grows linearly in n.

Asymptotic run times don't make sense when talking about inputs of fixed or bounded length. We always have to consider *arbitrarily long* inputs when using asymptotics. This is because $\mathsf{poly}(n) = T$ is a polynomial for any T. If inputs are at most 1000 bits long, say, then even an algorithm $\mathcal{A}$ running in time $T = 2^{2^{1000}}$ would be considered "efficient."

2.4 Efficiency of arithmetic

Running a $\mathsf{poly}(n)$-time subroutine $\mathsf{poly}'(n)$ times takes $\mathsf{poly}(n) \cdot \mathsf{poly}'(n)$ time, which itself is polynomially bounded (Proposition 2.1). This means that we can build up a library of PPT algorithms using poly-time subroutines without concerning ourselves too much about figuring out exact time complexity.

Integer addition, subtraction, multiplication, division (over $\mathbb{Z}$): Reading, printing, adding, and subtracting n-bit numbers can be done in $O(n)$ time. Multiplying and dividing

[5] A word of caution. Usually, one can change the order of universal quantifiers without trouble: $\forall a, \forall b, f(a,b) \iff \forall b, \forall a, f(a,b)$. But "for all sufficiently large" hides an existential quantifier $(\exists n_0)$ inside it. "For all a, for all sufficiently large n, $f(a,n)$" is NOT equivalent to "for all sufficiently large n, for all a, $f(a,n)$." See Definition 2.4 for an example.

[6] The maximum is over all length n inputs and all possible random coin flips of the algorithm. Another way of formalizing polynomial time for randomized algorithms is to require that the *expected* (as in expectation) running time is bounded above by a polynomial. Requiring that the running time is always bounded instead of bounded in expectation is a bit simpler, and we'll brush the differences between these formalisms under the rug. For more information, go read about Las Vegas algorithms and Monte Carlo algorithms.

n-bit numbers can be done in $O(n^2)$ time (actually, even faster). We will treat these as primitive operations.

Modular reduction, $f : x \mapsto x \bmod m$ Assume $m > 0$. The simplest algorithm is the one you were probably taught in discrete math. The goal is to find the value $0 \le r < m$ such that $x = km + r$. Compute $k = \lfloor \frac{x}{m} \rfloor$, then solve for $r = x - km$ (all over the integers). This uses just a few primitive integer operations. This means that modular addition and multiplication can be done in polynomial time by first computing over $\mathbb{Z}$ then reducing modulo m.

2.4.1 Brute-force factoring by trial division

Our factoring assumption (Section 2.7) says that no PPT algorithm can factor random Blum integers N except with negligible probability. You may be wondering: Can't we factor N by just trying to divide by $2, 3, \ldots, \sqrt{N}$? You can, and it will take $O(\sqrt{N})$ divisions.

$\mathcal{A}(N)$ – Trial division (not PPT)

for $d = 2, 3, \ldots, \lfloor \sqrt{N} \rfloor$ **do**
  if $d \mid N$ **then return** d
return $\perp$

That's **not polynomial time**! The running time of algorithms is measured in terms of the **length** of the input: the number of bits it takes to write it down. The value N takes $\|N\| \approx \log N$ bits to write down. The numerical value N is exponential in this quantity: $N = 2^{\log N}$. In the worst case (when $p, q \approx \sqrt{N}$) trial division requires on the order of $\sqrt{N} \approx 2^{\|N\|/2}$ divisions in the worst case. Thus the running time is **exponential** in the input length, even before accounting for the cost of each division.

2.4.2 Printing n bits

Here's an easy problem: print an n-bit number.

- Input: $n \ge 1$
- Output: $x \in \{0,1\}^n$

Seems easy!

$\mathcal{A}(n)$ – not PPT

return $x = 0^n$

The above algorithm is **not PPT**! $\mathcal{A}(n)$ runs in time n, which is the time it takes to write its n-bit output 0^n. But the input to $\mathcal{A}(n)$ is $\|n\| \approx \log n$ bits long! Like with brute force factoring, the run time of $\mathcal{A}$ is exponential in its input length: $n \approx 2^{\|n\|}$.

This is a technicality, but one we must address. There is **no PPT algorithm** for the problem as stated above. To restate the problem to allow for a PPT algorithm, we change

the input to make its length n instead of logarithmic in n. The convention for how we do this is to write down n in **unary** instead of binary.

- INPUT: 1^n
- OUTPUT: $x \in \{0,1\}^n$

Now the input n is written as a string of n ones. The length of 1^n is n. So a PPT algorithm can run in time $\mathsf{poly}(n)$ instead of $\mathsf{poly}(\log n)$.

$\mathcal{A}(1^n)$ – **PPT**

return $x = 0^n$

2.4.3 Modular exponentiation

Let's analyze the complexity of computing $f : (a, x, m) \mapsto a^x \bmod m$, for $a \in \mathbb{Z}$, $x \in \mathbb{N}$, and $m > 0$. Exponentiation over $\mathbb{Z}$ ($(a, x) \mapsto a^x$) is **not** PPT. This is because the output a^x takes exponentially many bits to write down in the worst case. For example, 2^x takes $x + 1$ bits to represent. As we said above, x is exponential in its bit length $\log x$. Modular exponentiation doesn't have this problem. Its output is in the range $[0, m)$ – values which can be represented in fewer bits than the input itself.

A naive algorithm performs $O(x)$ modular multiplications (each multiplication itself has a polynomial cost in the bit length of m):

$$a^x \equiv \underbrace{a \cdot a \cdot a \ldots a \cdot a}_{x \text{ times}} \mod m.$$

As with the $O(\sqrt{N})$-time algorithm for factoring, this is not polynomially-bounded.

Luckily, there is a polynomial time algorithm for modular exponentiation, called **repeated squaring**. For $x > 0$, let $n = \|x\| \approx \log x$.

- Compute the binary representation of x. These are the values $z_0, z_1, \ldots, z_n \in \{0,1\}$ such that $x = \sum_{i=0}^{n} z_i \cdot 2^i$.
- Compute the first $n + 1$ powers of 2 of a modulo m:

  $$a \equiv a^{2^0} \mod m, \quad a^2 \equiv a^{2^1} \mod m, \quad a^4 \equiv a^{2^2} \mod m, \quad \ldots, \quad a^{2^n} \mod m.$$

  Each term can be computed with one multiplication (squaring the previous term) and one modular reduction.

- Compute $a^x \bmod m$ by computing the product

  $$\prod_{i:z_i=1} a^{2^i} \equiv \prod_{i=0}^{n} a^{z_i \cdot 2^i} \equiv a^{\sum_{i=0}^{n} z_i \cdot 2^i} \equiv a^x \mod m.$$

  This requires at most $n + 1$ multiplications and modular reductions.

Each of the three steps in this algorithm requires $O(n)$ poly-time operations, so the whole thing can be done in poly-time.

Example 2.2. To compute $3^{13} \bmod 7$, write $13 = 8+4+1 = 0b1101$. Repeated squaring gives

$$3^1 \equiv 3 \bmod 7, \qquad 3^2 \equiv 2 \bmod 7, \qquad 3^4 \equiv 4 \bmod 7, \qquad 3^8 \equiv 2 \bmod 7.$$

Use the powers corresponding to the three 1-bits:

$$3^{13} \equiv 3^8 \cdot 3^4 \cdot 3^1 \equiv 2 \cdot 4 \cdot 3 \equiv 3 \bmod 7.$$

2.5 Negligible functions

For run times and efficient algorithms, we think about functions that grow to infinity: $f(n) \to \infty$ as $n \to \infty$. When analyzing success probabilities, we instead think about functions that go to 0: $\epsilon(n) \to 0$ as $n \to \infty$.

2.5.1 Motivation

An example of an NP-hard problem is the Travelling Salesman Problem (TSP): given a complete weighted graph $G = (V, E)$, find the shortest cycle that visits every vertex exactly once. Consider an algorithm $\mathcal{A}$ that, on input graph G, outputs a random permutation of the vertices. $\mathcal{A}$ is clearly efficient. It runs in time polynomial in the length of the graph's encoding. Because there are at most $|V|!$ cycles visiting each vertex in the graph exactly once, the probability of outputting the shortest cycle is at least $\frac{1}{|V|!} > 0$.

Why don't we consider this an efficient algorithm for TSP? It's PPT and sometimes gets the right answer! Because $\frac{1}{|V|!}$ is tiny. For example, for $|V| = 101$, $\frac{1}{|V|!} \approx 10^{-160}$. That's an unthinkably tiny number. Here's a thought experiment. The number of atoms in the universe is estimated to be about 10^{80}. If every atom contained inside it another universe with just as many sub-atoms as ours, there would be about 10^{160} of those sub-atoms. If you and I each choose one sub-atom uniformly at random, the chance we pick the same one is about $\frac{1}{101!}$. What the example illustrates is that when considering randomized algorithms, small probabilities of success can't be avoided (it can get lucky!).

Even though small probabilities of success can't be avoided, it is sometimes possible to treat them as effectively 0 for all practical purposes. More concretely, suppose an algorithm $\mathcal{A}$ running in time T had a probability $\epsilon(n)$ of succeeding in some task, where n is the security parameter. To increase the success probability, one could iteratively run $\mathcal{A}$ many times. Let $\mathcal{B}_t$ be the algorithm which runs $\mathcal{A}$ iteratively $t(n)$ times, outputting the result of the first iteration that succeeds. We can bound $\mathcal{B}_t$'s success probability using the Union Bound:

$$\Pr[\mathcal{B}_t \text{ succeeds}] = \Pr[\bigvee_{i=1}^{t} \text{ iteration } i \text{ succeeds}] \leq t\epsilon.$$

For security against **all PPT adversaries**, we need security against $\mathcal{B}_t$ for any polynomial number of iterations $t(n)$. We need to be able to choose the value of the security parameter

n so that $\epsilon(n)$ is as small as we want. That is, we want $t(n) \cdot \epsilon(n) \to 0$ as $n \to \infty$ for any polynomial t.

Such functions are called **negligible functions**.

2.5.2 Definition

Negligible functions go to zero *faster than any inverse polynomial.*

> **Definition 2.4.** *A non-negative function $f : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is **negligible** if for all $c \geq 0$*
>
> $$f(n) = O\left(\frac{1}{n^c}\right).$$
>
> *Otherwise, f is* non-negligible. *We denote a generic negligible function by* negl.

Equivalently, f is negligible if and only if any of the following hold:

- For all $c > 0$, for all sufficiently large n, $f(n) < 1/n^c$.
- For all $c > 0$, $\lim_{n\to\infty} f(n) \cdot n^c = 0$.

The inverse of any positive polynomially-bounded function is *non-negligible.* For example, n^{-2} or $1/(n \log n)$. The reverse is not true: it is possible for $f(n)$ to not be polynomially-bounded, and for $1/f(n)$ to be non-negligible. For example (see Exercise 2.4):

$$f(n) = \begin{cases} 1 & n \text{ even} \\ 2^n & n \text{ odd} \end{cases}.$$

Still, the most common negligible functions we will encounter are those of the form $1/f(n)$ for some $f(n)$ that is clearly not poly: 2^{-n}, $2^{-\sqrt{n}}$, $n^{-\log n}$, or $1/n!$.

Negligible functions are eventually smaller than any inverse-polynomial. An inverse-polynomial can be smaller than a negligible function for some inputs. For example, consider $\mathsf{negl}(n) = 100 \cdot 2^{-n}$ and $\mathsf{poly}(n) = n^2$. For $n = 5$, we have $\mathsf{negl}(5) > 3$ and $1/\mathsf{poly}(5) = 1/25$. But this can only be true for finitely many inputs. *For all sufficiently large inputs n,* $\mathsf{negl}(n) \leq 1/\mathsf{poly}(n)$, provided $\mathsf{poly}(n) > 0$ for sufficiently large n.

2.6 Useful properties of poly and negl functions

One of the reasons asymptotic analysis is so common is that polys and negls are so easy to work with. Once you get used to them, you can usually think of them in fairly crude terms: polys combined with polys are generally poly; negligibles combined with negligibles are generally negligible; negligibles "kill" polys.

These are called closure properties. The following proposition lists useful cases of closure properties that come up frequently. Applying them should become second nature.

Proposition 2.1. *Let* $\mathsf{poly}, \mathsf{poly}' : \mathbb{N} \to \mathbb{N}$ *be arbitrary polynomially-bounded functions. Let* $\mathsf{negl}, \mathsf{negl}'$ *be arbitrary negligible functions.*

(a) $f_1(n) = \mathsf{poly}(n) + \mathsf{poly}'(n)$ *is polynomially-bounded.*

(b) $f_2(n) = \mathsf{poly}(n) \cdot \mathsf{poly}'(n)$ *is polynomially-bounded.*

(c) $f_3(n) = \mathsf{poly}'(\mathsf{poly}(n))$ *is polynomially-bounded.*

(d) $f_4(n) = \mathsf{negl}(n) + \mathsf{negl}'(n)$ *is negligible.*

(e) $f_5(n) = \mathsf{poly}(n) \cdot \mathsf{negl}'(n)$ *is negligible.*

Proof. See Exercise 2.5. □

2.7 The Factoring Assumption (Blum integers)

Given a PPT algorithm for sampling primes, there is a PPT algorithm for sampling Blum integers. Recall that a Blum integer is a number N which is the product of two distinct primes $N = pq$ where p and q are both congruent to 3 mod 4.

Let $\mathsf{Primes}(1^n)$ sample a uniformly random n-bit prime. The algorithm $\mathsf{Blum}(1^n)$ below samples independent pairs of such primes until they are distinct and both congruent to 3 mod 4, then returns their product. We treat these samplers as PPT, suppressing the technicalities described in the footnote.[7]

For sufficiently large n, the output $N = pq$ has $2n - 1$ or $2n$ bits, and is uniform over the set of products of two distinct n-bit primes congruent to 3 mod 4. Every eligible ordered pair (p, q) is equally likely, and every product has exactly two such representations, (p, q) and (q, p). This specifies the distribution used in our factoring assumption.

Notice that Primes and Blum take their input in unary ($\mathsf{Blum}(1^n)$) rather than binary ($\mathsf{Blum}(n)$). This is for the reasons explained in Section 2.4.2.

$\mathsf{Blum}(1^n)$

$p \leftarrow \mathsf{Primes}(1^n)$, $q \leftarrow \mathsf{Primes}(1^n)$ independently
while $(q = p)$ *OR* $(p \not\equiv 3 \bmod 4)$ *OR* $(q \not\equiv 3 \bmod 4)$ **do**
  $p \leftarrow \mathsf{Primes}(1^n)$, $q \leftarrow \mathsf{Primes}(1^n)$ independently
return pq

Here n is the security parameter and measures the bit length of the factors, rather than the product. Because $\|N\| \in \{2n - 1, 2n\}$, polynomial running time and negligible success probability can equivalently be measured in n or in the bit length of N.

[7] To sample a prime, repeatedly sample a uniform n-bit integer and test whether it is prime. Primes have density on the order of $1/n$, so this takes polynomial time in expectation. Requiring both primes to be congruent to 3 mod 4 and distinct still takes polynomial time in expectation. To meet our strict PPT definition, cap the number of trials (e.g., $\leq n^5$), aborting and failing if that number is exceeding. The failure probability and any primality-testing error can be made negligible. We suppress these details and describe the ideal samplers with exact primality tests. For the finitely many small n admitting fewer than two eligible primes, define $\mathsf{Blum}(1^n) = 21$; the pseudocode below covers the remaining cases.

Assumption 2.1 (Factoring Blum Integers (FACTORING))
For any PPT algorithm $\mathcal{A}$, there exists a negligible function negl *such that for all sufficiently large n,*

$$\Pr_{N,\mathcal{A}}\Big[\big(1 < d < N\big) \text{ and } \big(d \text{ divides } N\big)\Big] < \mathsf{negl}(n),$$

in the probability experiment: $N \leftarrow \mathsf{Blum}(1^n)$, $d \leftarrow \mathcal{A}(N)$.

The probability above is taken over the sampling of $N \leftarrow \mathsf{Blum}(1^n)$, and also the randomness of the algorithm $\mathcal{A}$. Notice that the left and right sides of the inequality are *both* functions of n. The dependence on n on the left side comes from the definition of the probability experiment.

The fastest known algorithm is called the Generalized Number Field Sieve. Its time complexity is on the order of $2^{n^{1/3}(\log n)^{2/3}}$. A list of factoring world records can be found here: `https://en.wikipedia.org/wiki/Integer_factorization_records`. An 896-bit semiprime was factored on September 19, 2026 by an Anthropic employee using Claude to help write the GNFS code and orchestrate its execution on 2048 GPUs over 10 days, using a total of about 30 GPU-years of compute time.[8]

2.8 Exercises

Exercise 2.1. Determine whether each of the following algorithms is PPT or not. Explain your answers. Throughout, $x \in \{0,1\}^n$ and $n > 0$. We identify $\{0,1\}^n$ with the integers $\{0, 1, \ldots, 2^n - 1\}$ in the natural way.

Hint: Three are PPT, one is not.

(a) $\mathcal{A}_1(x)$

$z = 0$
for $i = 1, \ldots, n$ **do**
  $z := z + 1$
return z

(b) $\mathcal{A}_2(x)$

$z = 0$
for $i = 1, \ldots, 2^{10000}$ **do**
  $z := z + 1$
return z

[8] `https://saweis.net/posts/rsa-896.html`

(c) $\mathcal{A}_3(x)$

$z = 0$
for $i = 1, \ldots, x$ **do**
  $z := z + 1$
return z

(d) $\mathcal{A}_4(1^x)$

$z = 0$
for $i = 1, \ldots, x$ **do**
  $z := z + 1$
return z

Exercise 2.2. Determine whether each of the following algorithms is PPT for all PPT $\mathcal{A}$ or not. Explain your answers. If $\mathcal{B}$ is not PPT for all PPT $\mathcal{A}$, give a counterexample.

Throughout, $x \in \{0,1\}^n$ and $n > 0$. We identify $\{0,1\}^n$ with the integers $\{0, 1, \ldots, 2^n - 1\}$ in the natural way.

Hint: Two are PPT, one is not.

(a) $\mathcal{B}_1(x)$

return $\mathcal{A}(x)$

(b) $\mathcal{B}_2(x)$

for $i = 1, \ldots, n$ **do**
  $z_i := \mathcal{A}(x)$
return $(z_1, z_2, \ldots, z_n)$

(c) $\mathcal{B}_3(x)$

$z_0 := x$
for $i = 1, \ldots, n$ **do**
  $z_i := \mathcal{A}(z_{i-1})$ // *Notice $\mathcal{A}$'s input changed from previous question*
return $(z_1, z_2, \ldots, z_n)$

Exercise 2.3. Let $f : \mathbb{N} \to \mathbb{R}_{\geq 0}$ be a function. Prove that f is negligible if and only if There exists a negligible function negl such that for all sufficiently large n,

$$f(n) < \mathsf{negl}(n).$$

Exercise 2.4. Show that the following function is neither polynomially bounded nor negligible:

$$f(n) = \begin{cases} 1 & n \text{ even} \\ 2^n & n \text{ odd} \end{cases}.$$

Exercise 2.5. Prove Proposition 2.1

Exercise 2.6 (Success by repetition)**.** Suppose that Illinois scratch-off lottery tickets win with probability $\epsilon(n)$, independently of all other tickets. You plan to buy $t(n)$ tickets to guarantee at least $1/4$ probability of finding a winner:

$$\Pr[\text{at least 1 ticket of } t(n) \text{ tickets wins}] \geq 1/4.$$

Hint: Write an exact expression for the above probability in terms of ϵ and t. Solve for t.

(a) Let $\epsilon = 1/100$. What is the minimum number of tickets t you must buy?

Hint: For the following, simplify using the inequality $1 - x \leq e^{-x}$ which holds for all $x \in \mathbb{R}$.

(b) Let $\epsilon(n) = n^{-5}$. Show that the number of tickets $t(n)$ you must buy is polynomially-bounded.

(c) Let $\epsilon(n) = 2^{-n}$. Show that the number of tickets $t(n)$ you must buy is not polynomially-bounded.

Exercise 2.7 (Related to the discussion preceding Def 2.4)**.** For $n > 0$, let p_n^* be the largest n-bit prime number. Let $\mathsf{isPrime}$ be a PPT algorithm that returns TRUE if its input is prime.

Suppose $\mathcal{A}$ is a PPT algorithm satisfying the following: there exists a polynomial poly, such that for all n, there exists a number $1 \leq i \leq n$ such that

$$\Pr[\mathcal{A}(1^n, i) = p_n^*] \geq \frac{1}{\mathsf{poly}(n)}.$$

(a) Describe a PPT algorithm $\mathcal{B}$ in pseudocode such that:

- $\mathcal{B}$ uses $\mathcal{A}$ as a subroutine.
- $\Pr[\mathcal{B}(1^n) = p_n^*] \geq \frac{1}{\mathsf{poly}(n)}$.

Justify your answer.

(b) Describe a PPT algorithm $\mathcal{B}$ in pseudocode such that:

- $\mathcal{B}$ uses $\mathcal{A}$ as a subroutine.
- $\mathcal{B}$ makes exactly 1 subroutine call to $\mathcal{A}$.
- $\Pr[\mathcal{B}(1^n) = p_n^*] \geq \frac{1}{\mathsf{poly}'(n)}$ for some polynomial poly'.

Justify your answer.

Exercise 2.8 (★). This problem is based on the excerpt from [Katz and Lindell, 2021] at the beginning of these notes. It helps motivate the computational approach to defining security for encryption schemes. Specifically, the problem below shows that effective but inefficient *known-plaintext attacks* are always possible, and that even efficient attacks can succeed with tiny probability.

Let $(\mathsf{Enc}, \mathsf{Dec})$ be an encryption scheme with message space $\mathcal{M}$ and keyspace $\mathcal{K} = \{0,1\}^n$, for some $n > 0$.

Consider an adversary $\mathcal{A}$ who gets as input $z > 0$ plaintext-ciphertext pairs $(m_1, c_1), \ldots, (m_z, c_z)$ all encrypted using the same (unknown) key k^*. $\mathcal{A}$ also gets a *challenge* ciphertext c^*. The plaintext for c^* is $m^* = \mathsf{Dec}_{k^*}(c^*)$. $\mathcal{A}$ is not given m^*.

$\mathcal{A}(c^*, m_1, c_1, m_2, c_2, \ldots, m_z, c_z)$

Initialize $m' \leftarrow \perp$
for $i = 1, \ldots, t$ **do**
  Sample $k' \xleftarrow{\$} \mathcal{K}$ uniformly at random without replacement
  if $\left(\mathsf{Dec}_{k'}(c_1) = m_1 \ \mathit{AND}\ \mathsf{Dec}_{k'}(c_2) = m_2 \ \mathit{AND}\ \ldots \ \mathit{AND}\ \mathsf{Dec}_{k'}(c_z) = m_z\right)$ **then**
    return $m' = \mathsf{Dec}_{k'}(c^*)$
return $\perp$

(a) Let $t = 1$. Show that for all $k^* \in \mathcal{K}$ and for all $(m_1, c_1), \ldots, (m_z, c_z), (m^*, c^*)$ as above:

$$\Pr[\mathcal{A} \text{ returns } m^*] > 0.$$

For the rest of the problem, let $t = 2^n$. We will show that when z is large enough,

$$\Pr[\mathcal{A} \text{ returns } m^*] > 1/2.$$

(b) What is $\Pr[\mathcal{A} \text{ returns } \perp]$?

Suppose that $m, m_1, \ldots, m_z, m^*$ are independently and identically distributed over $\mathcal{M}$. Assume that each encryption, including the encryption producing c^*, uses fresh independent randomness.

For $k', k^* \in \mathcal{K}$, let

$$q(k', k^*) := \Pr[\mathsf{Dec}_{k'}(\mathsf{Enc}_{k^*}(m)) = m],$$

where the probability is over the random choice of m and the randomness of Enc.

(c) Fix k^*. Let $F \subseteq \mathcal{K}$ be the set of *"far keys"* for k^* defined as $F = \{k' : q(k', k^*) < \frac{7}{8}\}$. Show the following: *(Hint: Use a Union Bound over the keys.)*

$$\Pr[\exists k' \in F \text{ s.t. } \left(\mathsf{Dec}_{k'}(c_1) = m_1 \wedge \mathsf{Dec}_{k'}(c_2) = m_2 \wedge \cdots \wedge \mathsf{Dec}_{k'}(c_z) = m_z\right)] \leq 2^n \cdot \left(\frac{7}{8}\right)^z.$$

The probability is over the random choice of the m_i and the randomness of Enc.

(d) Find z_0 polynomial in n such that for all $z \geq z_0$:

$$2^n \cdot \left(\frac{7}{8}\right)^z < \frac{1}{10}.$$

(e) Let $t = 2^n$. Using the previous parts, show that for any $k^* \in \mathcal{K}$ and $z \geq z_0$:

$$\Pr[\mathcal{A} \text{ returns } m^*] > \frac{3}{4}.$$

Hint: Analyze the probability that $\mathcal{A}$ returns m^* in two cases — when the key that passes the IF condition is in F, and when it's not. Use the Law of Total Probability to stitch it all together.

Bibliography

[Katz and Lindell, 2021] Katz, J. and Lindell, Y. (2021). *Introduction to Modern Cryptography*. Chapman and Hall/CRC, 3rd edition.